



T.C.
SAKARYA ÜNİVERSİTESİ

BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
PROGRAMLAMA DİLLERİNİN PRENSİPLERİ ÖDEV RAPORU

ŞEHİR SİMÜLASYON

B231210582 - Lucas Isaac Cassoma KATALAHALI

SAKARYA

Nisan, 2026

Programlama Dillerinin Prensipleri Dersi

ŞEHİR SİMÜLASYON

Lucas Isaac Cassoma/Katalahali^{B231210582} 1A

Özet

Bu proje, Nesne Yönelimli Programlama prensipleri doğrultusunda geliştirilmiş hiyerarşik bir şehir simülasyon sistemidir. Sistem; şehir, ilçe, mahalle ve kişi katmanlarından oluşmaktadır. Simülasyonun temel amacı, nüfus artışını modellemek ve belirli bir eşiği (1000 kişi) aşan birimlerin dinamik olarak bölünmesini sağlamaktır. Projede hiyerarşik yapıların yönetimi için soyutlama ve modüler sınıflar kullanılmıştır. Java diliyle geliştirilen sistemde, rastgele veri üretimi için Java Faker kütüphanesinden yararlanılmıştır. Bu çalışma, karmaşık veri yapılarının OOP ile yönetimini ve sorumlulukların ayrılması prensibini somutlaştırmıştır.

© 2026 Sakarya Üniversitesi.

Bu rapor benim özgün çalışmamdır. Faydalanmış olduğum kaynakları içerisinde belirttim. Her hangi bir kopya işleminde sorumluluk bana aittir.

Anahtar Kelimeler: OOP, Simülasyon, Veri Yapıları, Java

1. GELİŞTİRİLEN YAZILIM

Sistemdeki tüm idari birimler (Şehir, İlçe, Mahalle) bir yeri temsil ettiği için **Yer** adlı soyut bir sınıftan türetilmiştir. Bu soyut sınıf; isim ve nüfus gibi ortak özellikleri ve metotları içermektedir. Her biri nüfus hesaplama metodunu kendi yapısına göre implemente etmektedir. Nüfus yönetimi en alt seviyede (mahalle) gerçekleştirilmekte, üst birimlerin nüfusu ise bağlı oldukları alt birimlerin toplamı ile hiyerarşik olarak hesaplanmaktadır.

Sistemde aynı ilçeye bağlı mahallelerin nüfusları her zaman eşittir. Bu nedenle, mahalle seviyesinden başlayarak bir ilçenin toplam nüfusu, mahalle sayısı ile bir mahallenin nüfusunun çarpılmasıyla hesaplanmaktadır. Aynı mantık şehir için de geçerlidir. Bir şehrin nüfusu, ilçe sayısı ile bir ilçenin nüfusunun çarpılmasıyla hesaplanmaktadır.

Oyun sınıfı, programın ana mantığını içeren sınıftır. Bu sınıfta şehir listesi oluşturulmaktadır. Şehir sınıfı ilçe listesini, ilçe sınıfı mahalle listesini ve mahalle sınıfı ise kişi listesini içermektedir. Böylece şehirden başlanarak sistemin en alt seviyesine kadar erişim sağlanabilmektedir. Her şehir, sahip olduğu nüfus ile temsil edilmektedir. Simülasyon başlamadan önce kullanıcıdan alınan sayıların geçerli olup olmadığı kontrol edilmektedir. Sorumlulukların karışmaması için **Düzenleyici** adlı ayrı bir sınıf oluşturdum. Bu sınıf, kullanıcıdan alınan verilerin düzenlenmesi ve doğrulanmasından sorumludur. Ayrıca artış oranı gibi sistemde kullanılan bazı hesaplamalar da bu sınıf içerisinde gerçekleştirilmektedir. Bu sınıftaki metotlar **statik** olarak tanımlanmıştır. Böylece bu metotları kullanabilmek için sınıftan bir nesne oluşturmaya gerek kalmamaktadır. Kullanıcı tarafından girilen değerler geçersiz ise, bu sınıf tarafından uygun hale getirilmektedir. Böylece veri kontrolü ve hesaplama işlemleri ana oyun mantığından ayrılmış olur. Kontroller tamamlandıktan sonra simülasyon başlatılmaktadır.

Her turda mahallelerdeki nüfus belirli bir oranda artmaktadır. Bunun için şehir listesi **for-each** döngüsü ile gezilmektedir. Her şehir için ilçeler, her ilçe için mahalleler döngüye alınmaktadır. Her turda, mahalledeki mevcut kişiler önce bir yaş yaşlanmakta, daha sonra ise mahalleye yeni kişiler eklenmektedir. En içteki döngü

tamamlandıktan sonra üst seviyedeki nüfuslar güncellenmektedir. Örneğin, mahalleler güncellendikten sonra ilçe nüfusu, ilçeler güncellendikten sonra şehir nüfusu güncellenmektedir. Her turun sonunda şehirlerin nüfusu kontrol edilmektedir. Eğer bir şehrin nüfusu dört basamaklı bir sayıya ulaşırsa, şehir ikiye bölünmektedir. Bu işlemin uygulanması belirli kurallara bağlıdır. Öncelikle şehir listesi gezilerek nüfusu 1000 veya üzeri olan şehirler geçici bir listeye alınmaktadır. Daha sonra bu liste üzerinden işlem yapılarak yeni şehirler oluşturulmaktadır. Böylece döngü sırasında ana liste değiştirilmemiş olur. Bu bölme işlemi için üç farklı yöntem geliştirdim:

İlçe Bazlı Bölme: Eğer şehir birden fazla ilçeye sahipse, ilçe sayısı ikiye bölünmektedir. Bölme işlemi tam sayı bölmesi olduğu için yeni şehrin ilçe sayısı bu sonuca göre belirlenmektedir. Eğer bölme tam değilse, eski şehir daha fazla ilçeye sahip olmaktadır. İlçelerin nüfusları eşit olduğu için ilk ilçeler alınarak yeni şehre eklenmektedir.

Mahalle Bazlı Bölme: Eğer şehirde yalnızca bir ilçe varsa, bölme mahalle sayısına göre yapılmaktadır. Yeni şehir de bir ilçeye sahip olmaktadır. Mahalle sayısı ikiye bölünerek yeni şehrin mahalle sayısı belirlenmektedir.

Nüfus Bazlı Bölme: Eğer şehirde yalnızca bir ilçe ve bir mahalle varsa, bölme doğrudan nüfus üzerinden yapılmaktadır. Mahallenin nüfusu ikiye bölünür ve yeni mahalle oluşturularak yeni ilçe ve yeni şehir oluşturulmaktadır.

2. ÇIKTILAR

Kullanıcıdan sistem başlangıcında şehir sayıları alınmaktadır. Örneğin kullanıcı tarafından girilen değerler: 10 22 34 55.

Bu değerler sistemde doğrudan kullanılmadan önce Düzenleyici sınıfı tarafından kontrol edilmektedir.

```
Sehir numaralarini boslukla ayirarak girin: 10 22 34 55

Başlangıç Durum:
[11]-[22]-[36]-[55]
```

Her tur sonunda, nüfusu 1000 veya üzerine çıkarak dört basamaklı hale gelen şehirler otomatik olarak ikiye bölünür. Örneğin; başlangıçta 11 nüfuslu bir şehir, 3. tur sonunda iki ayrı şehre bölünür.

```
Tur sayisini girin: 3
Sehir numaralarini boslukla ayirarak girin: 11

==== 3. Tur ====

[704]-[704]
```

Son turda kullanıcı tarafından seçilen koordinata göre ilgili şehrin detaylı bilgileri ekrana yazdırılmaktadır:

```
[22]

Bir şehir hakkındaki bilgileri görmek için:
Satır girin (0'dan başlayarak): 0
Sutun girin (0'dan başlayarak): 0
Şehir: Van - Nüfus: 22
İlçe: Edirne - Nüfus: 22
Mahalle: Batuhan Port - Nüfus: 22
Kisiler:
    1 - Mehmet Özdemir - 29
    2 - Batuhan Zengel - 14
    3 - İrem Özdemir - 31
    4 - Mehmet Sağdıç - 21
    5 - Yiğit Ekkaldır - 26
    6 - Burak Özdemir - 5
    7 - Yiğit Ekkaldır - 17
    8 - Yiğit Davut - 39
    9 - Buse Sağdıç - 10
   10 - Caner Eren - 17
   11 - Caner Davut - 47
   12 - Yiğit Ekkaldır - 43
   13 - İrem Özkanlı - 43
   14 - İrem Zengel - 22
   15 - Mehmet Özkanlı - 33
   16 - Mehmet Zengel - 14
   17 - Mehmet Eren - 23
   18 - Batuhan Sağdıç - 33
   19 - Buse Eren - 10
   20 - İrem Zengel - 42
   21 - Batuhan Özdemir - 45
   22 - Batuhan Eren - 45

Oyunu kapatmak için bir tusa yada Enter tusuna basın...
```

3. SONUÇ

Bu proje, nesne hiyerarşisinin ve sorumlulukların ayrılması (Separation of Concerns) prensibinin pratikteki önemini göstermiştir. Soyut sınıfların kullanımı kod tekrarını önlemiş, modüler yapı ise sistemin sürdürülebilirliğini artırmıştır. Çalışma sonucunda, büyük veri yapılarının hiyerarşik bir düzende nasıl yönetileceği ve dinamik liste operasyonlarının güvenli şekilde nasıl yürütüleceği konusunda önemli deneyimler kazanılmıştır.

Referanslar

- [1] Java Faker Library Documentation: <https://github.com/DiUS/java-faker>
- [2] Doç. Dr. M. Fatih ADAK (MFA) channel: <https://www.youtube.com/watch?v=nCvipKIDndM&t=17s>